Assistente Inteligente de Filas de Atendimento

Fila virtual com ESP32 simulada no Wokwi, previsão de espera pelo vizinho mais próximo (1-NN) e envio dos dados por MQTT

Disciplina: Arquitetura e Organização de Computadores

Professor: Sandro A. Ferraz

Aluno: Lucas Barreto Santana · RM 573149 · Turma 1CCPG

Projeto integrador: Ponto Vivo (Citizen Assist IA)

Projeto no Wokwi: https://wokwi.com/projects/480447178434064385

Código no GitHub: github.com/Dustyluc123/ponto-vivo-citizen-assist-ai (pasta arquitetura)

Entrega: novembro de 2026

1. Problema

Em postos de atendimento, bibliotecas e secretarias, o cidadão chega sem saber quantas pessoas estão na frente nem quanto tempo vai esperar. No Ponto Vivo, o assistente que o passageiro acessa pelo QR code do ponto de ônibus, isso aparece num caso concreto: quem precisa resolver o Bilhete Único num posto de atendimento não sabe se vale a pena ir agora. O projeto simula uma **fila virtual**: botões registram entrada e atendimento, o ESP32 estima a espera com uma IA simples baseada em exemplos e publica os números pela internet, para que um painel ou o próprio assistente possam mostrá-los. Os tempos usados são fictícios e não representam um serviço público real.

2. Componentes

Qtd.	Componente	Ligação	Função
1	ESP32 DevKit C	—	Processa, guarda os dados e se comunica por Wi-Fi
2	Botões (ENTRADA verde, ATENDER azul)	GPIO 18 e 19, outro terminal no GND	Somar uma pessoa / concluir um atendimento
1	LCD 16×2 com módulo I²C	SDA GPIO 21, SCL GPIO 22, 5 V, GND	Mostrar fila e espera estimada
2	LEDs verde e vermelho + resistores de 220 Ω	GPIO 25 e 26	Fila curta (até 6) ou longa (7 ou mais pessoas)
—	Wokwi, Arduino/C++, bibliotecas LiquidCrystal I2C e PubSubClient	—	Simular e programar

3. Diagrama de arquitetura

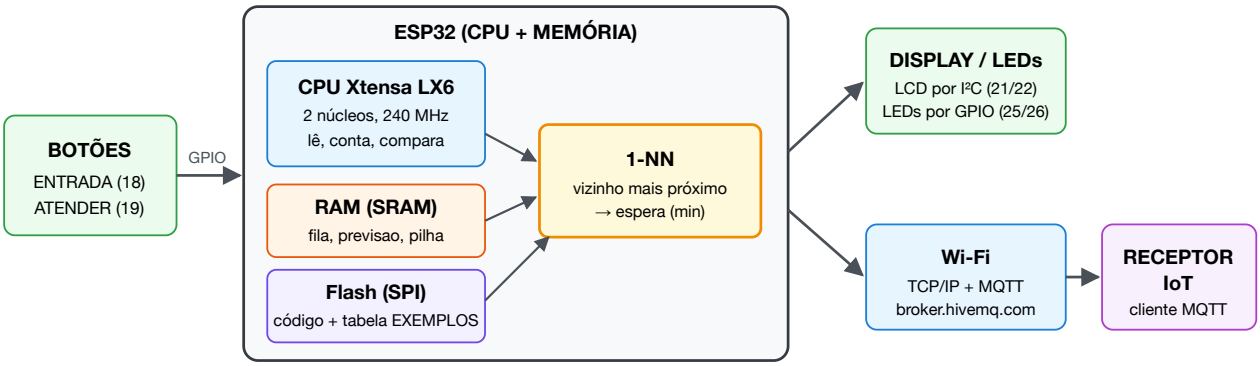


Figura 1. BOTÕES → ESP32 (CPU + MEMÓRIA) → 1-NN → DISPLAY/LEDs e Wi-Fi → RECEPTOR IoT.

4. Como CPU, memória e entrada/saída cooperam

Entrada / GPIO

Cada botão liga um pino de GPIO ao GND. O pino é configurado com `INPUT_PULLUP`: um resistor interno o mantém em 3,3 V (nível lógico HIGH) com o botão solto; ao apertar, o pino é ligado a 0 V (LOW). A CPU lê esse nível no registrador de entrada do GPIO com `digitalRead()`. Como o contato mecânico "quica" e gera várias transições em poucos milissegundos, o programa só aceita uma mudança depois que a leitura fica igual por 50 ms (**debounce** medido com `millis()`, sem travar o programa) e só conta a borda de descida: segurar o botão vale um aperto.

CPU

O ESP32 tem uma CPU Xtensa LX6 de 2 núcleos a 240 MHz (confirmado no serial: `ESP32-D0WDQ6-V3, 2 nucleos, 240 MHz`). O sistema FreeRTOS deixa a pilha de Wi-Fi num núcleo e a função `loop()` do programa no outro. A cada volta do `loop()` a CPU busca e executa as instruções que: (1) leem os dois botões; (2) somam ou subtraem 1 da fila, recusando valores negativos; (3) percorrem os 6 exemplos calculando subtrações, valores absolutos e comparações (busca 1-NN, O(n)); (4) mandam os resultados para o LCD, os LEDs, o serial e o Wi-Fi.

RAM e Flash

O ESP32 tem 520 KB de SRAM interna e, no DevKit, 4 MB de memória Flash externa ligada por SPI. O programa imprime ao ligar o endereço de cada dado, o que mostra a separação na prática:

```
[RAM]   variavel fila       em 0x3FFC3A2C     (SRAM de dados)
[RAM]   struct previsao     em 0x3FFC3A24
[FLASH] tabela EXEMPLOS     em 0x3F410AC5     (Flash mapeada para dados constantes)
[FLASH] funcao preverEspera em 0x400D27C4     (Flash mapeada para instruções)
[RAM]   heap livre: 283172 bytes
```

- **RAM:** o contador `fila`, a última `previsao` e o estado dos botões são variáveis globais que mudam o tempo todo. Os resultados temporários do 1-NN (`distancia`, `resultado`) ficam na **pilha** e somem quando a função termina. O heap guarda os buffers do Wi-Fi e do MQTT. Tudo na RAM se perde ao reiniciar.
- **Flash:** o código compilado e a tabela `EXEMPLOS`, declarada `const`, ficam na Flash: não mudam, não ocupam RAM e continuam lá depois de desligar. A CPU os lê através de uma cache, por isso aparecem em endereços como `0x3F4`… (dados) e `0x400D`… (instruções).

Saída / I²C e LEDs

O LCD usa um módulo I²C (expansor PCF8574, endereço `0x27`). O ESP32 é o **mestre** do barramento de dois fios: SCL (GPIO 22) leva o relógio e SDA (GPIO 21) leva os dados. Cada caractere é enviado como bytes endereçados ao 0x27, e o expansor converte esses bytes nos sinais paralelos que o controlador do LCD entende. Os LEDs são mais simples: `digitalWrite()` escreve no registrador de saída do GPIO, que coloca 3,3 V no pino; o resistor de 220 Ω limita a corrente.

Wi-Fi

O rádio de 2,4 GHz do ESP32 conecta à rede `Wokwi-GUEST`, que dá acesso à internet. A pilha TCP/IP abre uma conexão TCP com o broker `broker.hivemq.com` na porta 1883 e o programa envia um pacote **MQTT PUBLISH** com o JSON. O broker repassa a mensagem a todos os clientes que assinam o tópico.

5. Algoritmo 1-NN

A previsão usa exemplos guardados em vez de uma fórmula fixa. Para a fila atual, o programa calcula a distância `|fila − pessoas do exemplo|` para cada um dos 6 exemplos e devolve o tempo do mais próximo.

Pessoas na fila	0	2	4	6	8	10
Tempo observado (min)	0	6	12	18	25	32

> **Critério de desempate:** com número ímpar de pessoas, dois exemplos ficam à mesma distância (com 5 pessoas, os de 4 e 6). O programa escolhe o de **maior tempo** (5 pessoas → 18 min). É uma estimativa pessimista de propósito: para o cidadão, ser atendido antes do previsto é melhor do que esperar mais do que foi prometido. O serial registra cada empate.

Fora da faixa: acima de 10 pessoas o vizinho mais próximo é sempre o exemplo de 10 (32 min), então o valor deixa de ser confiável. Nesse caso o display mostra `Espera: +32 min` alternando com `Fora da faixa!`, o serial avisa que a espera real deve ser maior e o JSON leva `"fora_da_faixa": true`.

6. Comunicação IoT

A cada mudança da fila o ESP32 publica, por MQTT, no tópico `fiap/gs2026/pontovivo/fila01`:

```
{"dispositivo":"fila01","pessoas_na_fila":4,"espera_min":12,"fora_da_faixa":false}
```

A mensagem vai com a opção *retain*, então um painel que se conectar depois recebe o último estado. A conexão não bloqueia a fila: se o Wi-Fi ou o broker caírem, os botões continuam funcionando, o programa tenta reconectar a cada 5 s e reenvia o estado atual quando volta. O recebimento foi conferido no cliente MQTT da HiveMQ (`hivemq.com/demos/websocket-client`), assinando o tópico `fiap/gs2026/pontovivo/#`.

7. Testes

Teste	Ação	Valor esperado	Valor obtido	Evidência
1	Iniciar o sistema	Fila = 0; espera = 0	LCD `Fila: 0 pessoas / Espera: 0 min`, LED verde aceso	Fig. 2
2	ENTRADA 4 vezes	Fila = 4; 12 min	LCD `Fila: 4 pessoas / Espera: 12 min`	Fig. 3
3	ATENDER uma vez	Fila = 3; nova previsão	`Fila: 3 pessoas / Espera: 12 min` (empate 2 e 4, fica o maior)	Fig. 4
4	ATENDER com fila vazia	Fila continua 0	Fila 0; serial `ATENDER ignorado: fila vazia`	Fig. 5
5	Chegar a 8 pessoas	LED de fila longa	`Espera: 25 min`, LED vermelho aceso e verde apagado	Fig. 6
6	Mudar a fila com Wi-Fi ativo	Mensagem recebida no serviço IoT	JSON recebido no cliente MQTT da HiveMQ a cada mudança	Fig. 7

Teste extra: com 11 pessoas o display mostrou `Espera: +32 min` alternando com `Fora da faixa!` e o serial registrou o aviso. O limiar de fila longa é de **7 pessoas**, porque a partir daí a espera estimada passa de 20 minutos (7 pessoas já dão 25 min pelo critério de desempate).

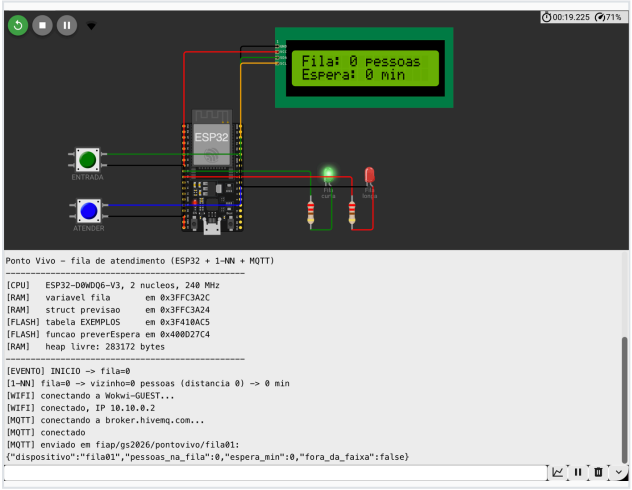


Figura 2. Teste 1: sistema iniciado, fila 0 e espera 0.

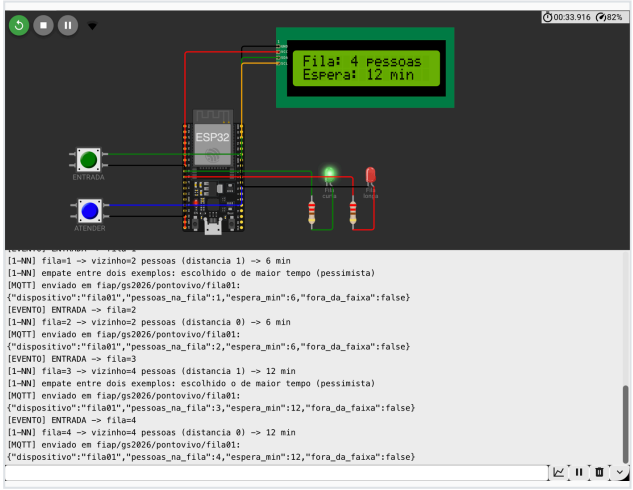


Figura 3. Teste 2: quatro entradas, 12 min.

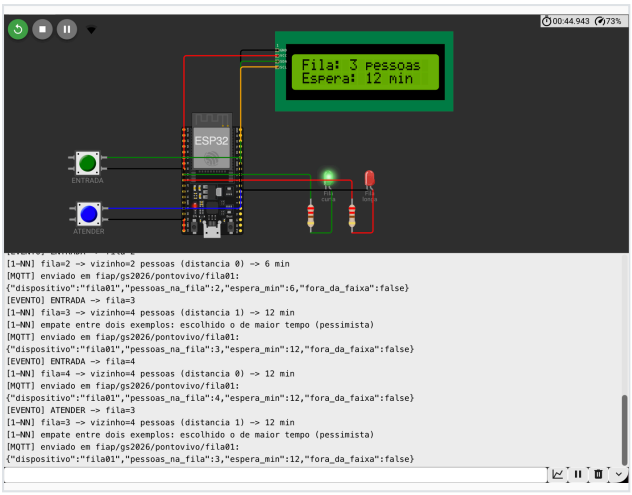


Figura 4. Teste 3: um atendimento, fila 3.

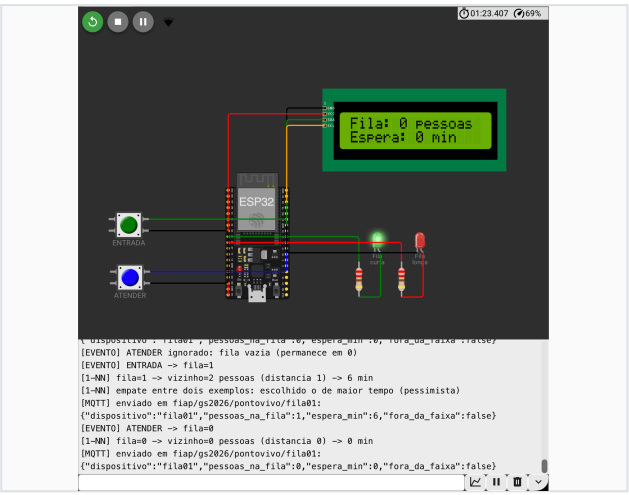


Figura 5. Teste 4: ATENDER com fila vazia.

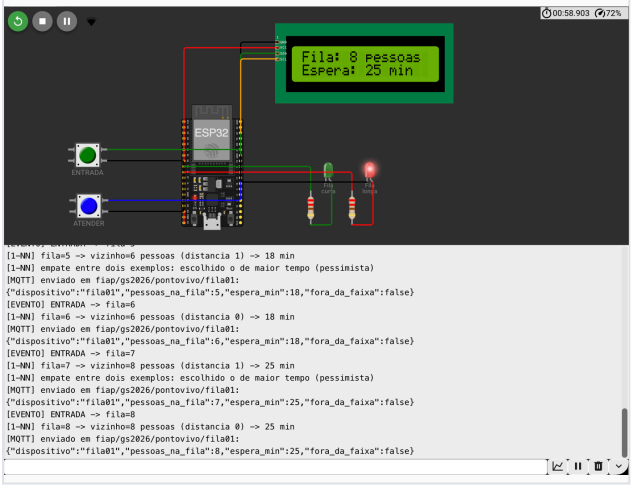


Figura 6. Teste 5: 8 pessoas, LED vermelho.

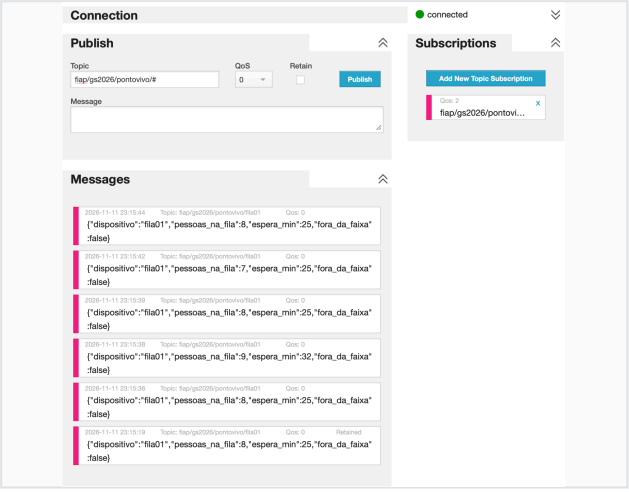


Figura 7. Teste 6: mensagens recebidas no cliente MQTT.

8. Conclusão

O sistema cumpre o funcionamento pedido: conta a fila sem permitir valores negativos, trata o repique dos botões, estima a espera com 1-NN e um critério de desempate explícito, sinaliza fila longa nos LEDs e publica cada mudança por MQTT, com recebimento conferido num serviço externo. Do ponto de vista da arquitetura, o projeto mostra a divisão de trabalho dentro do microcontrolador: o GPIO transforma o aperto em nível lógico, a CPU executa a contagem e a busca, a RAM guarda o que muda, a Flash guarda o que é fixo, e o I²C e o Wi-Fi levam o resultado para fora.

Limitações e melhorias: os tempos da tabela são fictícios; com tempos medidos de verdade e mais exemplos, a previsão ficaria mais fiel. A fila se perde se o ESP32 reiniciar (poderia ser gravada na memória não volátil). O broker público não tem senha nem criptografia, então um uso real exigiria um broker próprio com TLS. No Ponto Vivo, o mesmo módulo poderia contar quem espera no ponto de ônibus e alimentar o assistente e o planejamento da frota.